



Ministerio del Poder Popular para la Educación

Universidad Nacional Experimental de Guayana

Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

Sección 2

Tema 4: Analisis Lexico

Profesor:

Marquez Felix

Estudiantes:

Cortez Samuel V27.158.842

Duarte Alejandro V28.530.657

Moreno Alondra V30.331.8710

Perdomo Arnaldo V30.791.551

Ciudad Guayana, Julio 2026

Introducción.....	3
Actividad 1.....	5
Definición Formal de un AFD.....	5
Definición Formal de un Autómata de Pila (PDA).....	6
Ejemplos Prácticos.....	7
Comparación (Importancia vs AFD/AFND).....	8
Conclusión.....	8
Actividad 2.....	9
Analizador Léxico para Dockerfile mediante Expresiones.....	9
Diseño Del Analizador Léxico (Lexer).....	9
Tabla de Tokens y Patrones Regex Definidos.....	9
Paso a Paso de la Construcción del Lexer.....	12
Paso 1: Definición del Alfabeto y Componentes Léxicos.....	12
Paso 2: Ensamblado de Expresiones Regulares en un Solo Patrón.....	12
Paso 3: Mecanismo de Escaneo (Bucle Principal).....	13
Código Fuente Del Lexer.....	13
Pruebas Y Resultados De Ejecución.....	17
Prueba 1: Archivo Dockerfile Válido (Dockerfile_ejemplo1).....	17
Contenido de Entrada.....	17
Salida Real Lexer (PowerShell).....	18
Prueba 2: Archivo Dockerfile con Error Léxico (Dockerfile_ejemplo2).....	19
Contenido de Entrada.....	19
Salida del Lexer (PowerShell).....	20
Prueba 3: Archivo Dockerfile con Comentarios y Espacios en Blanco (Dockerfile_ejemplo3).	
20	
Contenido de Entrada.....	20
Salida del Lexer (PowerShell).....	20
Actividad 3: Construir un lexer para L utilizando metacompilador.....	21
1. Manual de Usuario de Flex (Metacompilador).....	21
2. Descripción del Lenguaje Diseñado.....	22
Codigo fuente rust_lexer.l.....	23
Codigo fuente pruebas.rs.....	25
Actividad 4 — Lenguajes de Seguridad Informática y FLEX.....	26
Herramientas de seguridad que usan Flex.....	26
Lenguaje elegido: reglas de Snort (archivo .rules).....	26
Tokens del lenguaje.....	27
Conclusión.....	30
Bliografía.....	32

Introducción

El análisis léxico constituye la fase fundamental en el proceso de compilación de un lenguaje de programación, actuando como el puente entre el código fuente escrito por el desarrollador y las estructuras de datos que las fases posteriores del compilador podrán interpretar. Este proceso, también conocido como tokenización, se encarga de leer una secuencia de caracteres y agruparlos en unidades significativas denominadas *tokens*, asignándoles un tipo y un valor (*lexema*), a la vez que detecta y reporta errores en la estructura léxica del programa fuente.

El presente informe aborda de manera integral los conceptos teóricos y las aplicaciones prácticas del análisis léxico, en concordancia con los lineamientos establecidos para el Tema 4 de la unidad curricular Lenguajes y Compiladores. El documento se estructura en cuatro actividades interrelacionadas que transitan desde la base teórica de las máquinas abstractas hasta la implementación de analizadores léxicos funcionales y su aplicación en entornos reales.

En primer lugar, se exploran los fundamentos de la teoría de autómatas, contrastando el poder expresivo de los Autómatas Finitos Deterministas (AFD), utilizados para reconocer lenguajes regulares (nivel 3 de la Jerarquía de Chomsky), con los Autómatas de Pila (PDA), capaces de reconocer lenguajes libres de contexto (nivel 2), sentando así las bases para comprender la separación entre el análisis léxico y el sintáctico. A continuación, se materializa la teoría mediante la construcción de un analizador léxico desde cero utilizando el lenguaje Python y expresiones regulares, aplicado específicamente a la verificación de archivos *Dockerfile*, demostrando un control total sobre el flujo de tokenización.

Posteriormente, se presenta un enfoque alternativo mediante el uso de metacompiladores, empleando la herramienta Flex para generar un lexer optimizado en C que reconoce un subconjunto del lenguaje Rust. Esta metodología evidencia cómo la automatización reduce significativamente el esfuerzo de codificación y minimiza la propensión a errores. Finalmente, la investigación se extiende al ámbito de la seguridad informática, analizando la aplicación de analizadores léxicos en herramientas como Snort y sistemas de detección de intrusiones, donde la velocidad y precisión en el procesamiento de reglas y logs son críticas para la detección de amenazas en tiempo real. A través de estas actividades, se busca consolidar los conocimientos adquiridos y destacar la versatilidad de los analizadores léxicos más allá del ámbito académico.

Actividad 1

Definición Formal de un AFD

Para comprender la necesidad de los Autómatas de Pila, recordemos primero la definición de un Autómata Finito Determinista (AFD), que es la base del Análisis Léxico.

Un AFD se define como una quintupla (5 elementos):

$$\text{AFD} = (Q, \Sigma, \delta, q_0, F)$$

Donde:

- Q : Conjunto finito de estados.
- Σ (Sigma): Alfabeto de entrada (símbolos que puede leer).
- δ (Delta): Función de transición. Se escribe como $\delta: Q \times \Sigma \rightarrow Q$. (Toma un estado y un símbolo, y devuelve UN SOLO estado).
- q_0 : Estado inicial ($q_0 \in Q$).
- F : Conjunto de estados de aceptación ($F \subseteq Q$).

Característica CLAVE del AFD (Determinista):

"Para cada estado actual y para cada símbolo del alfabeto que lea, siempre hay exactamente una (y solo una) transición posible."

Esto significa que no tiene memoria adicional, solo recuerda en qué estado está. Por ejemplo, un AFD no puede reconocer el lenguaje $a^n b^n$ (mismo número de 'a' que de 'b') porque no tiene forma de contar cuántas 'a' leyó. Esta limitación da paso a los Autómatas de Pila.

Definición Formal de un Autómata de Pila (PDA)

Un Autómata de Pila (*Pushdown Automaton*, PDA) se define como una séptupla (7 elementos), ya que añade componentes para manejar la memoria tipo pila:

$$\text{PDA} = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

A diferencia del AFD, aquí los elementos se desglosan de la siguiente manera:

Símbolo	Nombre	Explicación
Q	Conjunto finito de estados	Son todas las "posiciones" que puede tener la máquina. Ej: $\{q_0, q_1, q_2\}$.
Σ (Sigma)	Alfabeto de entrada	Símbolos que la máquina puede leer desde la cinta de entrada. Ej: $\{a, b, 0, 1\}$
Γ (Gamma)	Alfabeto de la pila	Símbolos que la máquina puede almacenar en la pila. Ej: $\{A, B, Z_0\}$.
δ (Delta)	Función de transición	Es la regla que define el movimiento. Se escribe formalmente como: $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$ (Explicación: Depende del estado actual, del símbolo leído (o ϵ = transición sin

		consumir carácter) y del tope de la pila. Devuelve un conjunto de posibles opciones (estado nuevo y nuevo tope de pila). El símbolo $P(\dots)$ significa que es no determinista, lo que le da el poder de explorar múltiples caminos para reconocer lenguajes libres de contexto).
q_0	Estado inicial	El estado donde la máquina empieza a leer ($q_0 \in Q$).
Z_0	Símbolo inicial de la pila	Es la "marca de fondo" que indica que la pila está vacía. Siempre está al inicio en el tope de la pila
F	Estados de aceptación	Subconjunto de Q. Si al finalizar la lectura la máquina termina en un estado de F (y la pila cumple la condición de vacío, según la definición), la cadena es aceptada.

Ejemplos Prácticos

Ejemplo 1: Lenguaje $a^n b^n$

Para reconocer cadenas con el mismo número de 'a' seguidas de 'b' (ej: aaabbb), el

PDA funciona así:

- **Estado q_0 :** Lee 'a', y apila el símbolo 'A' por cada 'a' leída.
- **Estado q_1 :** Al encontrar la primera 'b', cambia de estado y empieza a leer las 'b's. Por cada 'b' leída, desapila una 'A'.

- **Aceptación:** Si al terminar de leer la cadena, la pila queda vacía (solo tiene el símbolo inicial Z_0), la cadena es válida. Si sobran 'b' o sobran 'a', la pila no quedará vacía y se rechaza.

Ejemplo 2: Paréntesis balanceados (())

- Al leer un paréntesis de apertura (, se apila un símbolo (ej: P).
- Al leer un paréntesis de cierre), se desapila el símbolo P.
- Si al final de la lectura la pila está completamente vacía (solo queda el fondo Z_0), los paréntesis están balanceados. Un AFD no puede hacer esto porque no tiene memoria para saber cuántos paréntesis de apertura hay.

Comparación (Importancia vs AFD/AFND)

En la Jerarquía de Chomsky, los AFD y AFND reconocen los Lenguajes Tipo 3 (Regulares), mientras que los Autómatas de Pila (PDA) reconocen los Lenguajes Tipo 2 (Libres de Contexto - GLC). Esta diferencia es crucial porque los lenguajes de programación reales (como Rust, Python o C) tienen estructuras anidadas (if-else, bucles, funciones) que requieren memoria de pila para ser validadas. Por eso, el PDA es la base teórica del Análisis Sintáctico (Parser), que es el siguiente paso después del Análisis Léxico (donde se usan los AFD).

Conclusión

El autómata de pila es más poderoso que el finito porque añade memoria infinita (aunque con acceso restringido tipo pila LIFO), permitiendo reconocer lenguajes que requieren contar y anidar, características esenciales para la sintaxis de cualquier lenguaje de programación. Su capacidad no determinista (reflejada en la función de transición que

devuelve conjuntos de opciones) lo convierte en la herramienta teórica ideal para validar la estructura gramatical de un programa fuente.

Actividad 2

Analizador Léxico para Dockerfile mediante Expresiones

En el diseño de compiladores, el **Análisis Léxico** (o *tokenización*) constituye la primera fase de procesamiento del código fuente. Su función principal consiste en leer el flujo de caracteres de entrada, agruparlos en secuencias significativas denominadas *lexemas* y asociarles un tipo categórico llamado *token*.

Diseño Del Analizador Léxico (Lexer)

Para reconocer la estructura y los componentes léxicos de un archivo **Dockerfile**, se definió una lista de tokens utilizando Expresiones Regulares en Python. El orden de definición es crítico ya que el motor de expresiones regulares evalúa los patrones secuencialmente de arriba a abajo.

Tabla de Tokens y Patrones Regex Definidos

Token	Expresión Regular	Descripción / Semántica en Docker
FROM	\b(?:FROM from)\b	Define la imagen base desde la cual se construye el contenedor.
RUN	\b(?:RUN run)\b	Ejecuta comandos dentro del sistema de archivos del contenedor.
COPY	\b(?:COPY copy)\b	Copia archivos o directorios locales hacia la ruta del contenedor.

CMD	\b(?:CMD cmd)\b	Establece el comando por defecto al inicializar el contenedor.
EXPOSE	\b(?:EXPOSE expose)\b	Declara el puerto de red en el cual escuchará el contenedor.
ENV	\b(?:ENV env)\b	Define variables de entorno en el contenedor.
WORKDIR	\b(?:WORKDIR workdir)\b	Establece el directorio de trabajo activo del contenedor.
ARG	\b(?:ARG arg)\b	Define variables que los usuarios pueden pasar al constructor.
ADD	\b(?:ADD add)\b	Copia archivos (soporta URLs remotas y descompresión tar).
ENTRYPOINT	\b(?:ENTRYPOINT entrypoint)\b	Configura el contenedor para que corra como un ejecutable.
VOLUME	\b(?:VOLUME volume)\b	Crea un punto de montaje con acceso a almacenamiento externo.
USER	\b(?:USER user)\b	Establece el UID o nombre de usuario para ejecutar instrucciones.
LABEL	\b(?:LABEL label)\b	Añade metadatos organizacionales a la imagen.

NUMBER	<code>\b\d+(?:\.\d+)?\b</code>	Reconoce números enteros y de punto flotante (puertos, versiones).
STRING	<code>"[^"\\]*(?:\\.[^"\\]*)*"'/[^']*?(?:\\.[^']*)*'</code>	Cadenas de caracteres delimitadas por comillas simples o dobles.
LBRACKET	<code>[</code>	Corchete izquierdo de apertura <code>[</code> para arrays sintácticos.
RBRACKET	<code>]</code>	Corchete derecho de cierre <code>]</code> para arrays sintácticos.
COMMA	<code>,</code>	Carácter coma <code>,</code> separador de argumentos en arrays.
ASSIGN	<code>=</code>	Operador de asignación directa de valores.
ARGUMENT	<code>[a-zA-Z0-9_\\-\\.\\:\\@\\\$\\{\\}*\\+]+</code>	Rutas, tags de imágenes, variables de entorno u otros argumentos de comandos.
NEWLINE	<code>\n</code>	Salto de línea físico. Permite llevar el conteo exacto de líneas del archivo.
SKIP	<code>[\\t\r]+</code>	Caracteres de espacio, tabulaciones y retornos de carro (ignorados léxicamente).
COMMENT	<code>#[^\n]*</code>	Comentarios del desarrollador que inician con <code>#</code> (ignorados léxicamente).

MISMATCH	.	Atrapalotodo para cualquier carácter no especificado (levanta error léxico).
-----------------	---	--

Paso a Paso de la Construcción del Lexer

Para cumplir formalmente con la metodología del desarrollo teórico de un analizador léxico a partir de expresiones regulares, el proceso de construcción del software se dividió en tres fases secuenciales estructuradas de la siguiente manera:

Paso 1: Definición del Alfabeto y Componentes Léxicos

- Identificamos las directivas clave del archivo Dockerfile (como **FROM**, **RUN**, **COPY**, etc.) y las definimos como tokens independientes con sus respectivas palabras clave emparejadas con límites de palabra (**\b**) para evitar coincidencias parciales.
- Definimos tokens auxiliares como **NUMBER** (para puertos o versiones, soportando decimales), **STRING** (para rutas o textos entre comillas) y operadores especiales como **ASSIGN** (**=**), **LBRACKET** (**[**), **RBRACKET** (**]**) y **COMMA** (**,**) para arrays tipo **JSON** frecuentes en comandos.
- Definimos un token comodín **ARGUMENT** que capture rutas, tags, variables de entorno u otros valores compuestos de caracteres alfanuméricos y ciertos símbolos permitidos.

Paso 2: Ensamblado de Expresiones Regulares en un Solo Patrón

Para evaluar todas las reglas de forma eficiente en una sola pasada, unimos las expresiones regulares individuales utilizando el operador de unión OR (`|`) y asignamos a cada una un nombre de grupo de coincidencia (`(?P<NAME>pattern)`).

Paso 3: Mecanismo de Escaneo (Bucle Principal)

- Se utiliza la función `re.finditer` para buscar todas las ocurrencias del patrón combinado en el texto de entrada.
- En cada coincidencia, el motor Regex nos indica cuál fue la última subexpresión del grupo que coincidió (`mo.lastgroup`), determinando el tipo de Token.
- Se mantiene un control exacto de la posición: al encontrar un salto de línea (`NEWLINE`), se incrementa el contador de líneas (`line_num`) y se registra el desplazamiento inicial del inicio de línea actual (`line_start`) para calcular la columna exacta del token (`column = mo.start() - line_start`).
- Se ignoran los espacios (`SKIP`) y comentarios (`COMMENT`). Si el grupo coincidente es `MISMATCH`, se lanza una excepción de error léxico indicando la línea y el carácter no reconocido.

Código Fuente Del Lexer

```
import re

import sys

import os


# Definición de los tokens para instrucciones y componentes de
Dockerfile

tokens = [
```

```

('FROM', r'\b(?:FROM|from)\b'),

('RUN', r'\b(?:RUN|run)\b'),

('COPY', r'\b(?:COPY|copy)\b'),

('CMD', r'\b(?:CMD|cmd)\b'),

('EXPOSE', r'\b(?:EXPOSE|expose)\b'),

('ENV', r'\b(?:ENV|env)\b'),

('WORKDIR', r'\b(?:WORKDIR|workdir)\b'),

('ARG', r'\b(?:ARG|arg)\b'),

('ADD', r'\b(?:ADD|add)\b'),

('ENTRYPOINT', r'\b(?:ENTRYPOINT|entrypoint)\b'),

('VOLUME', r'\b(?:VOLUME|volume)\b'),

('USER', r'\b(?:USER|user)\b'),

('LABEL', r'\b(?:LABEL|label)\b'),

('NUMBER', r'\b\d+(?:\.\d+)?\b'),

('STRING',
r'"[^\\"]*(?:\\.["\\]*)*"|'\'[^\'\\]*(?:\\.['\\]*)*\''),

('LBRACKET', r'\['),

('RBRACKET', r'\]'),

('COMMA', r','),

('ASSIGN', r('='),

('ARGUMENT', r'[a-zA-Z0-9_\-\.\/\:\@\$\{\}\*\+]+'),

('NEWLINE', r'\n'),

('SKIP', r'[\t\r]+'),

('COMMENT', r'#[^\n]*'),

('MISMATCH', r'.')

```

```
]
```

```

def lexer(input_text):

    token_regex = '|'.join(f'(?P<{name}>{pattern})' for name, pattern
in tokens)

    line_num = 1

    line_start = 0

    for mo in re.finditer(token_regex, input_text, re.MULTILINE):

        kind = mo.lastgroup

        value = mo.group(kind)

        if kind == 'NEWLINE':

            line_start = mo.end()

            line_num += 1

        elif kind == 'SKIP' or kind == 'COMMENT':

            continue # Ignorar espacios, tabulaciones y comentarios

        elif kind == 'MISMATCH':

            raise RuntimeError(f'{value!r} unexpected on line
{line_num}')

        else:

            column = mo.start() - line_start

            yield kind, value, line_num, column

def cargar_archivo(nombre_archivo):

    try:

        with open(nombre_archivo, 'r', encoding='utf-8') as file:

            return file.read()

    except FileNotFoundError:

```

```

        print(f"Error: El archivo '{nombre_archivo}' no fue encontrado.
Sorry!")

        return None

    except Exception as e:

        print(f"Error al leer el archivo: {e}")

        return None

def analizar_y_mostrar(nombre_archivo):

    print(f"\n=====")

    print(f"Analizando archivo: {nombre_archivo}")

    print(f"=====\n")

    input_text = cargar_archivo(nombre_archivo)

    if input_text is None:

        return

    try:

        for token in lexer(input_text):

            print(token)

    except RuntimeError as e:

        print(f"ERROR LÉXICO: {e}")

if __name__ == "__main__":

    if len(sys.argv) > 1:

        # Analizar archivo pasado como argumento

        analizar_y_mostrar(sys.argv[1])

```

```
else:

    # Por defecto, analizar los tres archivos de prueba relativos
    al directorio del script

    dir_base = os.path.dirname(os.path.abspath(__file__))

    archivos_prueba = [

        os.path.join(dir_base, 'Dockerfile_ejemplo1'),

        os.path.join(dir_base, 'Dockerfile_ejemplo2'),

        os.path.join(dir_base, 'Dockerfile_ejemplo3')

    ]

    for archivo in archivos_prueba:

        analizar_y_mostrar(archivo)
```

Pruebas Y Resultados De Ejecución

Se diseñaron tres escenarios de prueba distintos para validar el correcto comportamiento del autómata léxico.

Prueba 1: Archivo Dockerfile Válido (Dockerfile_ejemplo1)

Contenido de Entrada

```
# Ejemplo 1: Dockerfile Valido

# Define la imagen base

FROM node:18-alpine

# Define variables de entorno
```

```
ENV NODE_ENV=production

ENV PORT=3000


# Establece el directorio de trabajo

WORKDIR /usr/src/app


# Copia los archivos de definicion de dependencias

COPY package*.json ./


# Ejecuta la instalacion de dependencias

RUN npm ci --only=production


# Copia el resto del codigo de la aplicacion

COPY . .


# Expone el puerto configurado

EXPOSE 3000


# Comando para iniciar la aplicacion

CMD ["node", "src/index.js"]
```

Salida Real Lexer (PowerShell)

```

=====
Analizando archivo: c:\MyProyectos\lenguaje-y-compiladores\Tema4\Actividad2_Docker\Dockerfile_ejemplo1
=====

('FROM', 'FROM', 3, 0)
('ARGUMENT', 'node:18-alpine', 3, 5)
('ENV', 'ENV', 6, 0)
('ARGUMENT', 'NODE_ENV', 6, 4)
('ASSIGN', '=', 6, 12)
('ARGUMENT', 'production', 6, 13)
('ENV', 'ENV', 7, 0)
('ARGUMENT', 'PORT', 7, 4)
('ASSIGN', '=', 7, 8)
('NUMBER', '3000', 7, 9)
('WORKDIR', 'WORKDIR', 10, 0)
('ARGUMENT', '/usr/src/app', 10, 8)
('COPY', 'COPY', 13, 0)
('ARGUMENT', 'package*.json', 13, 5)
('ARGUMENT', './', 13, 19)
('RUN', 'RUN', 16, 0)
('ARGUMENT', 'npm', 16, 4)
('ARGUMENT', 'ci', 16, 8)
('ARGUMENT', '--only', 16, 11)
('ASSIGN', '=', 16, 17)
('ARGUMENT', 'production', 16, 18)
('COPY', 'COPY', 19, 0)
('ARGUMENT', './', 19, 5)
('ARGUMENT', './', 19, 7)
('EXPOSE', 'EXPOSE', 22, 0)
('NUMBER', '3000', 22, 7)
('CMD', 'CMD', 25, 0)
('LBRACKET', '[', 25, 4)
('STRING', '"node"', 25, 5)
('COMMA', ',', 25, 11)
('STRING', '"src/index.js"', 25, 13)
('RBRACKET', ']', 25, 27)

```

Prueba 2: Archivo Dockerfile con Error Léxico (Dockerfile_ejemplo2)

Contenido de Entrada

```

# Ejemplo 2: Dockerfile con Errores Lexicos
FROM python:3.9-slim

# Directorio de trabajo
WORKDIR /app

# Intentamos usar un caracter no permitido en nuestro lexer (por
# ejemplo, un caracter unicode o simbolo especial no valido)
# @ no esta dentro de ningun patron valido fuera de comentarios, por lo
# tanto causara un error de token no reconocido
COPY . @dir

# Otra instruccion valida para continuar despues del error (si se
# recuperara, aunque levantara RuntimeError)
RUN pip install -r requirements.txt

# Simbolo de error adicional
%EXPOSE 8080

```

Salida del Lexer (PowerShell)

```
=====
Analizando archivo: c:\MyProyects\lenguaje-y-compiladores\Tema4\Actividad2_Docker\Dockerfile_ejemplo2
=====

('FROM', 'FROM', 2, 0)
('ARGUMENT', 'python:3.9-slim', 2, 5)
('WORKDIR', 'WORKDIR', 5, 0)
('ARGUMENT', '/app', 5, 8)
('COPY', 'COPY', 9, 0)
('ARGUMENT', '.', 9, 5)
('ARGUMENT', '@dir', 9, 7)
('RUN', 'RUN', 12, 0)
('ARGUMENT', 'pip', 12, 4)
('ARGUMENT', 'install', 12, 8)
('ARGUMENT', '-r', 12, 16)
('ARGUMENT', 'requirements.txt', 12, 19)
ERROR LÉXICO: '%' unexpected on line 15
```

Prueba 3: Archivo Dockerfile con Comentarios y Espacios en Blanco (Dockerfile_ejemplo3)

Contenido de Entrada

```
# Ejemplo 3: Dockerfile con Comentarios y Espacios
#
# Este archivo contiene principalmente comentarios y
# líneas en blanco para validar que el analizador lexico
# ignora estas partes correctamente y solo extrae la estructura util.
#
# Imagen base
FROM debian:stable-slim
#
# Espacios y tabulaciones antes de la directiva
ENV   APP_NAME="Docker Test"   VERSION=1.0
#
# Fin de la configuracion
# No hay mas comandos aqui.
```

Salida del Lexer (PowerShell)

```

=====
Analizando archivo: c:\MyProyectos\lenguaje-y-compiladores\Tema4\Actividad2_Docker\Dockerfile_ejemplo3
=====

('FROM', 'FROM', 8, 0)
('ARGUMENT', 'debian:stable-slim', 8, 5)
('ENV', 'ENV', 11, 6)
('ARGUMENT', 'APP_NAME', 11, 12)
('ASSIGN', '=', 11, 20)
('STRING', '"Docker Test"', 11, 21)
('ARGUMENT', 'VERSION', 11, 37)
('ASSIGN', '=', 11, 44)
('NUMBER', '1.0', 11, 45)

```

Actividad 3: Construir un lexer para L utilizando metacompilador

1. Manual de Usuario de Flex (Metacompilador)

Flex (Fast Lexical Analyzer Generator) es una herramienta de metacompilación diseñada para generar analizadores léxicos (lexers) de forma automatizada. Su funcionamiento se divide en tres fases fundamentales basadas en la teoría de autómatas:

1. **Especificación Léxica (.l):** El usuario define un archivo fuente con extensión .l. Este archivo se divide en tres secciones separadas por los delimitadores %%:
 - **Declaraciones:** Código en C nativo (librerías) y macros de expresiones regulares.
 - **Reglas de Transición:** Mapeo directo entre Expresiones Regulares (patrones) y Acciones en C (funciones printf para retornar tokens).
 - **Código de Soporte:** La función principal main() que inicia el flujo.
2. **Generación del Código Fuente en C:** Al ejecutar Flex sobre el archivo .l, el motor traduce cada expresión regular a una tabla de transiciones optimizada que imita el comportamiento de un Autómata Finito Determinista (AFD). El resultado es un archivo puro de lenguaje C llamado lex.yy.c.

3. **Lectura y Tokenización:** La función principal del analizador generado es `yylex()`, la cual abre el archivo fuente de entrada (`prueba.rs`) y lee la cadena de texto carácter por carácter. Cuando una cadena (lexema) coincide con una expresión regular, se activa su acción correspondiente y se reporta el componente léxico (token). Si un carácter no coincide con ninguna regla, es capturado por la regla por defecto . (punto), registrándose como un Error Léxico.

2. Descripción del Lenguaje Diseñado

El lenguaje **L** es un subconjunto estricto de la sintaxis del lenguaje de programación Rust, diseñado para reconocer estructuras básicas de funciones, declaraciones mutables, condicionales y ciclos.

A continuación, se presenta la especificación formal de la tabla de tokens implementada en nuestro archivo `rust_lexer.l`:

Componente Léxico (Token)	Expresión Regular (Regex)	Descripción / Ejemplos de Lexemas
PALABRA_RESERVADA	<code>fn, let, mut, if, else, loop, return, struct</code>	Palabras clave nativas del núcleo de Rust.
NUMERO_ENTERO	<code>[0-9]+</code>	Cadenas numéricas sin punto decimal (ej: 15, 10).
NUMERO_FLOTANTE	<code>[0-9]+\.[0-9]+</code>	Números con precisión decimal (ej: 3.14, 0.5).
IDENTIFICADOR	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	Nombres de variables o funciones creados por el usuario (ej: main, x).
LLAVE_ABIERTA	<code>{</code>	Delimitador de apertura de bloques de código.
LLAVE_CERRADA	<code>}</code>	Delimitador de cierre de bloques de código.
PARENTESIS_ABIERTO	<code>(</code>	Apertura de parámetros o condiciones.
PARENTESIS_CERRADO	<code>)</code>	Cierre de parámetros o condiciones.
FIN_SENTENCIA	<code>;</code>	Carácter delimitador de fin

		de instrucción.
OPERADOR_ASIGNACION	=	Símbolo de asignación mutativa.
COMENTARIOS	//... o /*...*/	Cadenas que se descartan del flujo de tokenización.
ERROR LÉXICO	.	Cualquier carácter que no pertenezca al alfabeto válido de \$L.

Código fuente rust_lexer.l

```
rust_lexer.l
1  %{
2  #include <stdio.h>
3  %}
4
5  %option noyywrap
6
7  DIGITO      [0-9]
8  LETRA       [a-zA-Z_]
9  IDENTIFICADOR {LETRA}({LETRA}|{DIGITO})*
10 ENTERO      {DIGITO}+
11 FLOTANTE     {DIGITO}+\.{DIGITO}+
12
13 %%
14
```

```

12
13 %%
14
15 "fn"      { printf("TOKEN: PALABRA_RESERVADA, Lexema: %s\n", yytext); }
16 "let"     { printf("TOKEN: PALABRA_RESERVADA, Lexema: %s\n", yytext); }
17 "mut"     { printf("TOKEN: PALABRA_RESERVADA, Lexema: %s\n", yytext); }
18 "if"      { printf("TOKEN: PALABRA_RESERVADA, Lexema: %s\n", yytext); }
19 "else"    { printf("TOKEN: PALABRA_RESERVADA, Lexema: %s\n", yytext); }
20 "loop"    { printf("TOKEN: PALABRA_RESERVADA, Lexema: %s\n", yytext); }
21 "return"  { printf("TOKEN: PALABRA_RESERVADA, Lexema: %s\n", yytext); }
22 "struct"  { printf("TOKEN: PALABRA_RESERVADA, Lexema: %s\n", yytext); }
23
24 {FLOTANTE} { printf("TOKEN: NUMERO_FLOTANTE, Lexema: %s\n", yytext); }
25 {ENTERO}   { printf("TOKEN: NUMERO_ENTERO, Lexema: %s\n", yytext); }
26 {IDENTIFICADOR} { printf("TOKEN: IDENTIFICADOR, Lexema: %s\n", yytext); }
27
28 "/*"([^\n"])* { /* Ignorar comentarios de una linea */ }
29 "/*"([^\n"]|\\"+["/*"])*\n/*" { /* Ignorar comentarios de bloque */ }
30
31 "{"        { printf("TOKEN: LLAVE_ABIERTA, Lexema: %s\n", yytext); }
32 "}"        { printf("TOKEN: LLAVE_CERRADA, Lexema: %s\n", yytext); }
33 "("        { printf("TOKEN: PARENTESIS_ABIERTO, Lexema: %s\n", yytext); }
34 ")"        { printf("TOKEN: PARENTESIS_CERRADO, Lexema: %s\n", yytext); }
35 ";"        { printf("TOKEN: FIN_SENTENCIA, Lexema: %s\n", yytext); }
36 "="        { printf("TOKEN: OPERADOR_ASIGNACION, %s\n", yytext); }
37
38 [ \t\n\r]+ { /* Ignorar espacios en blanco */ }
39 .          { printf("ERROR LEXICO: Caracter no reconocido: %s\n", yytext); }
40
41 %%

```

```

40
41 %%%
42
43 int main(int argc, char **argv) {
44     if (argc > 1) {
45         FILE *file = fopen(argv[1], "r");
46         if (!file) {
47             perror("Error al abrir archivo");
48             return 1;
49         }
50         yyin = file;
51     }
52     yylex();
53     return 0;
54 }

```

Codigo fuente pruebas.rs

```
prueba.rs x
prueba.rs > main
1 fn main() {
2     let mut x = 15;
3     // Comentario en Rust
4     if x == 10 {
5         return;
6     }
7 }
```

Salida de la Prueba

```
alejo@alejandro: /mnt/c/User x + v
TOKEN: LLAVE_CERRADA, Lexema: }
TOKEN: LLAVE_CERRADA, Lexema: }
alejo@alejandro:/mnt/c/Users/aleja/Documents/Univ
alejo@alejandro:/mnt/c/Users/aleja/Documents/Univ
a/Documents/Universidad/Semestre 7/Lenguajes y Co
alejo@alejandro:/mnt/c/Users/aleja/Documents/Univ
alejo@alejandro:/mnt/c/Users/aleja/Documents/Univ
t_lexer
alejo@alejandro:/mnt/c/Users/aleja/Documents/Univ
.rs
TOKEN: PALABRA_RESERVADA, Lexema: fn
TOKEN: IDENTIFICADOR, Lexema: main
TOKEN: PARENTESIS_ABIERTO, Lexema: (
TOKEN: PARENTESIS_CERRADO, Lexema: )
TOKEN: LLAVE_ABIERTA, Lexema: {
TOKEN: PALABRA_RESERVADA, Lexema: let
TOKEN: PALABRA_RESERVADA, Lexema: mut
TOKEN: IDENTIFICADOR, Lexema: x
TOKEN: OPERADOR_ASIGNACION, =
TOKEN: NUMERO_ENTERO, Lexema: 15
TOKEN: FIN_SENTENCIA, Lexema: ;
TOKEN: PALABRA_RESERVADA, Lexema: if
TOKEN: IDENTIFICADOR, Lexema: x
TOKEN: OPERADOR_ASIGNACION, =
TOKEN: OPERADOR_ASIGNACION, =
TOKEN: NUMERO_ENTERO, Lexema: 10
TOKEN: LLAVE_ABIERTA, Lexema: {
TOKEN: PALABRA_RESERVADA, Lexema: return
TOKEN: FIN_SENTENCIA, Lexema: ;
TOKEN: LLAVE_CERRADA, Lexema: }
TOKEN: LLAVE_CERRADA, Lexema: }
alejo@alejandro:/mnt/c/Users/aleja/Documents/Univ
```

Actividad 4 — Lenguajes de Seguridad Informática y FLEX

Herramientas de seguridad que usan Flex

Diversas herramientas del área de ciberseguridad emplean analizadores léxicos generados con Flex (o metacompiladores equivalentes) para procesar de forma rápida y confiable archivos de configuración, reglas de detección y registros de tráfico:

- Snort / Suricata (IDS/IPS): utilizan analizadores léxicos para parsear sus reglas de detección, almacenadas en archivos .rules.
- ModSecurity (WAF): tokeniza reglas SecRule para filtrar tráfico HTTP malicioso antes de que llegue a la aplicación.
- Analizadores de logs y SIEM (Apache/Nginx, Splunk): tokenizan líneas de log para extraer IPs, códigos de estado, rutas y otros campos relevantes.
- Escáneres de vulnerabilidades y disectores de protocolos (tcpdump, Wireshark): usan generadores de parsers similares para diseccionar el tráfico capturado.

Lenguaje elegido: reglas de Snort (archivo .rules)

Se seleccionó el lenguaje de reglas de Snort por ser uno de los ejemplos más representativos de tokenización aplicada a la seguridad informática. Una regla típica de Snort tiene la siguiente forma:

```
alert tcp any any -> 192.168.1.0/24 80 (msg:"Posible ataque SQL Injection";  
content:" OR '1'='1"; sid:1000001; rev:1;)
```

Tokens del lenguaje

Token	Ejemplo de lexema	Descripción
TK_ACTION	alert, log, pass, drop	Acción a tomar sobre el paquete
TK_PROTOCOL	tcp, udp, icmp, ip	Protocolo de red
TK_IP_ADDRESS	192.168.1.0, any	Dirección IP origen/destino (o comodín any)
TK_CIDR	/24	Máscara de red en notación CIDR
TK_PORT	80, 443, any	Puerto origen/destino
TK_DIRECTION	->, <>	Dirección del tráfico (unidireccional/bidireccional)
TK_KEYWORD_OPTION	msg, content, sid, rev, flags, pcre	Palabras clave dentro del cuerpo de la regla
TK_STRING	"Posible ataque SQL Injection"	Cadena literal (mensaje o patrón a buscar)
TK_NUMBER	1000001, 1	Valores numéricos (sid, revisión, etc.)
TK_SEPARATOR	;; ()	Separadores de opciones dentro de la regla
TK_COMMENT	# esto es un comentario	Líneas de comentario, ignoradas por el lexer

```
TK_ACTION(alert) TK_PROTOCOL(tcp) TK_IP_ADDRESS(any) TK_PORT(any)

TK_DIRECTION(->) TK_IP_ADDRESS(192.168.1.0) TK_CIDR(/24) TK_PORT(80)

TK_KEYWORD_OPTION(msg) TK_STRING("Posible ataque SQL Injection")

TK_KEYWORD_OPTION(content) TK_STRING("'" OR '1'='1")

TK_KEYWORD_OPTION(sid) TK_NUMBER(1000001)

TK_KEYWORD_OPTION(rev) TK_NUMBER(1)
```

Reflexión: ¿por qué Flex es útil en ciberseguridad?

Un analizador léxico generado con Flex resulta especialmente valioso en el ámbito de la seguridad informática por las siguientes razones:

- **Velocidad:** Flex genera autómatas finitos deterministas (AFD) altamente optimizados en C, capaces de procesar grandes volúmenes de paquetes o líneas de log por segundo, lo cual es crítico para inspeccionar tráfico en tiempo real sin introducir latencia.
- **Consistencia y confiabilidad:** al basarse en expresiones regulares formales, el reconocimiento de patrones es determinista, lo que reduce los falsos negativos frente a un parsing manual propenso a errores.
- **Separación de responsabilidades:** el lexer únicamente reconoce tokens (IPs, puertos, acciones, cadenas sospechosas), delegando en capas superiores —como el motor de reglas o la correlación de eventos— la decisión de si el patrón detectado representa una amenaza.
- **Detección de patrones maliciosos:** en un IDS como Snort, la tokenización rápida permite comparar el tráfico con firmas de ataques conocidos (inyección SQL, shellcode, escaneos de puertos) y generar alertas o bloqueos en tiempo real.

- Escalabilidad: al ser la primera fase de análisis (computacionalmente más económica) antes de técnicas más profundas como DPI (Deep Packet Inspection) o correlación de eventos, un lexer bien optimizado reduce la carga general del sistema de seguridad.

El uso de Flex en herramientas de seguridad permite construir la primera línea de defensa de forma eficiente: convertir flujos de datos crudos (tráfico, logs, reglas) en unidades léxicas estructuradas que los motores de análisis posteriores pueden interpretar y actuar sobre ellas con la rapidez que exige la detección de amenazas en tiempo real.

Conclusión

El desarrollo de las actividades contempladas en el presente informe ha permitido abordar de forma integral el análisis léxico, comprendiendo desde sus fundamentos matemáticos hasta su implementación práctica y sus aplicaciones en la industria.

En la primera actividad, se consolidó la comprensión de la teoría de autómatas, estableciendo una clara diferenciación entre los Autómatas Finitos (AFD/AFND) y los Autómatas de Pila. Se evidenció que, si bien los AFD son suficientes para reconocer los *tokens* que componen el análisis léxico (lenguajes regulares), su carencia de memoria los vuelve incapaces de manejar estructuras anidadas y conteos, como se demostró con los ejemplos clásicos de a^nb^n y los paréntesis balanceados. En contraste, los PDA, con su memoria de tipo pila (LIFO), sientan la base teórica indispensable para el análisis sintáctico (reconocimiento de lenguajes libres de contexto), demostrando su rol fundamental en la jerarquía de Chomsky y en la construcción de parsers para lenguajes de programación reales como Rust, Python o C.

La segunda actividad demostró la viabilidad de construir un analizador léxico desde cero utilizando expresiones regulares en Python. Se diseñó un lexer funcional para archivos *Dockerfile* con una tabla de tokens completa que abarca directivas (FROM, RUN), valores (NUMBER, STRING) y operadores (=, []). La implementación del bucle principal con `re.finditer` y el control preciso de líneas y columnas resultó ser una solución efectiva y didáctica, que además, mediante los estados de error (MISMATCH), logra detectar y reportar fallos léxicos de manera clara, como se reflejó en las pruebas de ejecución con archivos válidos y con errores.

En la tercera actividad, se exploró el enfoque de la metacompilación mediante la herramienta Flex. Se generó un analizador léxico altamente eficiente en C para un subconjunto del lenguaje Rust, demostrando que las herramientas automáticas reducen drásticamente la complejidad del desarrollo manual. La generación de tablas de transición basadas en autómatas finitos deterministas optimizados por Flex garantiza un procesamiento rápido y fiable, destacando la ventaja de este método para proyectos de gran envergadura donde el rendimiento es crítico.

Finalmente, la cuarta actividad amplió el horizonte del análisis léxico hacia el campo de la ciberseguridad. Se identificó que herramientas como Snort, Suricata y ModSecurity utilizan analizadores léxicos generados con Flex para tokenizar reglas de detección, logs de tráfico y paquetes de red. La reflexión realizada subraya que la velocidad de procesamiento, la consistencia en el reconocimiento de patrones y la capacidad de separar la identificación de tokens (firma de ataque) de la decisión lógica (acción a tomar) son cualidades esenciales para la detección de amenazas en tiempo real, validando así que los conceptos de compiladores trascienden el ámbito académico y son pilares en la infraestructura de seguridad moderna.

En síntesis, tanto el enfoque manual como el basado en metacompiladores son enfoques válidos y complementarios para la construcción de analizadores léxicos. La elección depende del contexto del proyecto: el enfoque manual ofrece control y comprensión profunda, mientras que el uso de herramientas como Flex proporciona velocidad, robustez y escalabilidad. Este trabajo ha permitido aplicar satisfactoriamente la teoría de autómatas a soluciones de software concretas, consolidando los conocimientos esenciales para el diseño de compiladores y su aplicación en diversos dominios de la ingeniería informática.

Bibliografía

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). *Introducción a la teoría de autómatas, lenguajes y computación* (3.^a ed.). Pearson Addison Wesley.
- Kelley, D. (1995). *Teoría de autómatas y lenguajes formales*. Prentice Hall.
- Linz, P. (2017). *An introduction to formal languages and automata* (6.^a ed.). Jones & Bartlett Learning.
- Sipser, M. (2013). *Introducción a la teoría de la computación* (3.^a ed.). Cengage Learning.
- Cisco Systems / Equipo Snort. (2026). Documentación de Snort y guía de escritura de reglas. Snort.org. <https://docs.snort.org/start/rules>
- Cisco Systems / Equipo Snort. (2026). Reglas de Snort y descarga del software IDS. Snort.org. <https://www.snort.org/snort3>
- Huntress. (2025). ¿Qué son las reglas de Snort? Conceptos básicos y beneficios. <https://www.huntress.com/cybersecurity-101/topic/snort-rules>
- Splunk. (2025). Reglas de Snort 101: ejemplos y casos de uso para la defensa de redes. https://www.splunk.com/en_us/blog/learn/snort-rules.html
- Cymulate. (2025). Reglas de Snort explicadas: estructura, sintaxis y casos de uso. <https://cymulate.com/cybersecurity-glossary/snort-rules/>
- OWASP Foundation. (2026). ModSecurity WAF. Guía para Desarrolladores de OWASP. <https://devguide.owasp.org/en/09-operations/03-modsecurity/>
- OWASP Foundation. (2024). Proyecto OWASP ModSecurity. <https://owasp.org/www-project-modsecurity/>

- OWASP ModSecurity. (2025). Manual de referencia (v3.x) [Wiki]. GitHub.
[https://github.com/owasp-modsecurity/ModSecurity/wiki/Reference-Manual-\(v3.x\)](https://github.com/owasp-modsecurity/ModSecurity/wiki/Reference-Manual-(v3.x))
- Proyecto Flex. (s.f.). Flex: el analizador léxico rápido. Proyecto GNU.
<https://westes.github.io/flex/manual/>
- **Márquez, F. (2026).** *Tema 4: Análisis Léxico*. Material didáctico de la cátedra de Lenguajes y Compiladores. Universidad Nacional Experimental de Guayana (UNEG).
- **Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008).** *Compiladores: Principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.
- **Levine, J. (2009).** *flex & bison: Text Processing Tools* (1.^a ed.). O'Reilly Media.
- **Louden, K. C. (2004).** *Construcción de compiladores: Principios y práctica*. Thomson Learning.